04. Type Casting

Created by Li Huanyu, last modified by suncaper on Mar 05, 2026

Introduction

In the previous lesson, we learned about Strings, Integers, and Floats. But what happens when you receive a number that is trapped inside a string? For example, `"100"`. Python sees the quotes and treats it as text. You cannot perform mathematical operations on text.

Type Casting is the process of forcing a variable to change from one data type to another. In Data Engineering, casting is a mandatory step in the "Transform" phase of an ETL pipeline before loading data into a database.

In this lesson, we will cover:

- **The Core Casting Functions** (`int()`, `float()`, `str()`).
- **Why it matters in Big Data** (Handling CSV/JSON inputs).
- **Common Casting Errors** (What happens when data is dirty).

Part 1: The Core Casting Functions

Python provides built-in functions to convert data types. They are named after the type you want to convert *into*.

1. Converting to Integer: `int()`

Removes decimals and converts numeric strings into whole numbers.

```
# From String to Integer
raw_id = "1045"
clean_id = int(raw_id)
print(clean_id * 2)  # Output: 2090 (Math works!)

# From Float to Integer (Warning: This cuts off the decimal!)
temperature = 98.6
rounded_temp = int(temperature)
print(rounded_temp)  # Output: 98
```

2. Converting to Float: `float()`

Adds decimal capabilities to integers or numeric strings.

```
# From String to Float
raw_price = "19.99"
clean_price = float(raw_price)
print(clean_price + 5.00)  # Output: 24.99

# From Integer to Float
base_score = 10
exact_score = float(base_score)
print(exact_score)  # Output: 10.0
```

3. Converting to String: `str()`

Turns any number back into text. You often need this when concatenating (joining) text and numbers together for a log message.

```
# ❌ Incorrect: Python cannot add a string and a number
age = 25
# print("User age is: " + age)  # TypeError!

# ✅ Correct: Cast the number to a string first
print("User age is: " + str(age))
```

Part 2: The "Big Data" Context (Why do we care?)

Imagine you are building a PySpark pipeline to process millions of rows from a CSV file.

Rule of Thumb: Every single piece of data pulled from a basic CSV is a `String`.

If you sort a dataset of prices while they are still Strings, the database will use "Alphabetical" sorting instead of "Numerical" sorting.

- **Numerical Sort (Correct):** `2, 9, 10, 100`
- **Alphabetical Sort (Wrong):** `"100", "10", "2", "9"` (Because "1" comes before "2" in the alphabet).

If you don't cast your columns to `int` or `float` before analyzing them, your reports will be completely wrong.

Part 3: Common Pitfalls & `ValueError` ❌

Python is smart, but it cannot perform miracles. If you try to cast something that doesn't make logical sense, Python will stop the program and throw a `ValueError`.

1. Text in a Number Function

```
user_input = "Apple"
# mapped_value = int(user_input)
# <--- ValueError: invalid literal for int() with base 10: 'Apple'
```

2. Floats trapped in an `int()`

If a string contains a decimal, you cannot convert it directly to an integer. You must convert it to a float first.

```
messy_string = "99.99"
# wrong_cast = int(messy_string)
# <--- ValueError: invalid literal for int()

# The Fix:
```

```
correct_cast = int(float(messy_string))
print(correct_cast) # Output: 99
```

Part 4: Hands-on Exercise (ETL Math Simulation)

Let's simulate processing a raw row of e-commerce data from a text file. We need to calculate the total order value, but the data types are wrong.

1. Open PyCharm.
2. Create a new file named `casting_practice.py`.
3. Type the following code:

```
# ---------------------------------------------------------
# Big Data Simulation: CSV Data Processing
# ---------------------------------------------------------

"""
Scenario: We read a line from a CSV file.
Everything is currently a string. We need to calculate the total cost.
"""

# 1. EXTRACT: Raw data from CSV
raw_item_name = "Wireless Mouse"
raw_price = "24.50"
raw_quantity = "3"

# 2. TRANSFORM: Cast types so we can do math
# Price needs decimals, Quantity is a whole number
clean_price = float(raw_price)
clean_quantity = int(raw_quantity)

# 3. APPLY LOGIC: Calculate total
total_cost = clean_price * clean_quantity

# 4. LOAD / OUTPUT: Create a log message
# We must cast the numeric result BACK to a string to print it nicely
log_message = "SUCCESS: Order for " + raw_item_name + " processed. Total: $" + str(total_cost)

print(log_message)
```

Run the code. Output:

```
SUCCESS: Order for Wireless Mouse processed. Total: $73.5
```

What happens?

1. We take `"24.50"` (text) and turn it into `24.5` (math-ready decimal).
2. We take `"3"` (text) and turn it into `3` (math-ready integer).
3. We multiply them to get `73.5`.
4. We take `73.5`, cast it back into a string `"73.5"`, and glue it to our success message.

Summary

- **`int()`** converts to a whole number (and drops decimals if converting from a float).
- **`float()`** converts to a decimal number.
- **`str()`** converts anything to text.
- **Data Pipelines depend on casting:** You must cast incoming text data into numbers before applying logic, sorting, or saving to a database.
- **`ValueError`** occurs if you try to cast non-numeric text into a number.

No labels